CƠ BẢN VỀ ĐỒ THỊ

Đồ thị (Graph):

- **Đồ thị (Graph)** G là một bộ đôi <V, E>, trong đó:
  - V: **tập các đỉnh** (vertex set), và
  - E: **tập các cung** (edge set), mỗi cung nối 2 đỉnh trong V

- Cho cung e nối 2 đỉnh x và y
  - Kí hiệu e = (x, y)
  - x, y được gọi là **đầu mút** (endpoints) của cung e
  - x và y được gọi là **kề nhau** (adjacent) hoặc **lân cận** của nhau (neighbours)
  - e được gọi là **liên thuộc** (incident) với x và y

- **Khuyên (loop)**: cung có hai đầu mút trùng nhau

- **Đa cung (multi edges)**: các cung có cùng chung đầu mút

Các loại đồ thị

- **Đơn đồ thị vô hướng (simple graph)**:
  - Cung không có hướng: (x, y) ≡ (y, x)
  - Không chứa khuyên
  - Không chứa đa cung

- **Đa đồ thị (Multigraphs)**
  - Cung không có hướng
  - Không chứa khuyên

- **Giả đồ thị (Pseudograph)**
  - Cung không có hướng
  - Có thể chứa đa cung và khuyên

- **Đơn đồ thị có hướng**
  - Cung có hướng (x, y) ≠ (y, x)
  - Không chứa chứa đa cung
    - Với mỗi cặp đỉnh x, y chỉ có thể tồn tại nhiều nhất 1 cung dạng (x, y) hoặc (y, x).

- **Đa đồ thị có hướng (không khuyên)**
  - Có thể chứa nhiều cung dạng (x, y)
  - Không chứa khuyên

- **Đa đồ thị có hướng (có khuyên)/quiver**
  - Có thể chứa nhiều cung dạng (x, y)
  - Có thể chứa khuyên

Bậc của đồ thị

- **Bậc (degree) của đỉnh**
  - Kí hiệu deg(x) = số cung liên thuộc với đỉnh x
    - Đỉnh có bậc = 0 gọi là đỉnh cô lập

- Đỉnh có bậc = 1 gọi là đỉnh treo

- **Định lý 1 (định lý bắt tay)**: Tổng bậc của tất cả các đỉnh trong một đồ thị = 2 lần số cung

$$\sum_{v \in V} \deg(v) = 2|E| = 2e$$

- **Định lý 2:** Số đỉnh bậc lẻ của một đồ thị vô hướng là số chẵn.

*** Đặc biệt (Đồ thị có hướng):**

- **Bậc vào của đỉnh**
  - Kí hiệu: $\deg^-(v)$ = số cung đi đến v
- **Bậc ra của đỉnh**
  - Kí hiệu: $\deg^+(v)$ = số cung đi ra từ v
- **Tính chất:**
  - $\deg(v) = \deg^-(v) + \deg^+(v)$
- **Định lý 3:**
  - Cho G = ⟨V, E⟩ làm một đồ thị có hướng

Đồ thị đặc biệt

- **Đồ thị đầy đủ (complete graph), Ký hiệu: K_n**
  - Đơn đồ thị vô hướng
  - Mỗi cặp đỉnh đều có đúng 1 cung

- **Đồ thị phân đôi (bipartite graph/bigraph), Ký hiệu: $K_{n,m}$**
  - Tập đỉnh được chia thành hai tập không giao nhau: U và V
  - Mỗi cung nối 1 đỉnh trong U và 1 đỉnh trong V

- **Đồ thị phân đôi đầy đủ**
  - Còn gọi là đồ thị 2 clique (biclique)
  - Mỗi đỉnh của phần này nối với tất cả các đỉnh của phần kia

- **Đồ thị đều bậc k (k-regular graph)**
  - Là đồ thị mà tất cả các đỉnh của nó đều có bậc k

- **Đồ thị vô hướng nền** của một đồ thị có hướng là đồ thị vô hướng có được sau khi đã loại bỏ hướng của các cạnh.

  - Đồ thị có hướng và đồ thị vô hướng nền của nó có cùng số cạnh

- **Đồ thị con**

- Cho đồ thị G = ⟨V, E⟩

  - Đồ thị con G_s = ⟨V, E_s⟩, $E_s \subset E$ được xây dựng từ G là đồ thị có được sau khi loại bỏ các cung không thuộc E_s.

• Đồ thị con $G_s = \langle V_s, E_s \rangle$, $V_s \subset V$, $E_s \subset E$ được xây dựng từ G là đồ thị có được sau khi loại bỏ các đỉnh không nằm trong V_s và các cung liên thuộc với nó, cùng với các cung không nằm trong E_s.

Đẳng cấu trong đồ thị

• 2 đồ thị $G_1 = \langle V_1, E_1 \rangle$ và $G_2 = \langle V_2, E_2 \rangle$ được gọi là **đẳng cấu** nếu và chỉ nếu tồn tại song ánh:

- $f: V_1 \rightarrow V_2$
- $v_2 = f(v_1) \in V_2$

sao cho $(x, y) \in E_1 \Leftrightarrow (f(x), f(y)) \in E_2$

Biểu diễn đồ thị

• **Cấu trúc dữ liệu đồ thị**
- Danh sách cung
- Ma trận kề (đỉnh – đỉnh)
- Ma trận đỉnh cung
- Danh sách kề (danh sách con/danh sách cha)

• **Các phép toán trên cấu trúc dữ liệu đồ thị:**
- init_graph(G): khởi tạo đồ thị
- adjacent(G, x, y): kiểm tra x và y có kề nhau không
- degree(G, x): tính bậc của đỉnh x
- neighbors(G, x): trả về danh sách các đỉnh kề của x
- add_edge(G, e, x, y): thêm cung e = (x, y) vào G
- remove_edge(G, e, x, y): xoá cung e = (x, y) ra khỏi G

Duyệt đồ thị

• **Đi qua tất cả đỉnh của một đồ thị**
- Cập nhật và/hoặc kiểm tra giá trị của chúng trên đường đi

• **Ý tưởng:**
- Bắt đầu từ 1 đỉnh bất kỳ, đánh dấu nó đã duyệt
- Duyệt các đỉnh kề của nó
- Duyệt các đỉnh kề của đỉnh kề của nó
- …

Phương pháp duyệt

* Duyệt theo chiều rộng (BFS)

• Sử dụng hàng đợi
• Các đỉnh trong hàng đợi là các đỉnh *đã duyệt* nhưng chưa xét các đỉnh kề của nó

• **Giải thuật (v1):**
- Đưa 1 đỉnh **s** bất kỳ vào hàng đợi (vd: đỉnh 1)
- **Duyệt s (vd: in s ra màn hình)**
- Đánh dấu **s** đã được duyệt

- **while** hàng đợi chưa rỗng **do**
  - Lấy đỉnh ở đầu hàng đợi ra ⇒ gọi là đỉnh u
  - **for** các đỉnh kề v của u **do**
    - **if** (v chưa được duyệt)
      - **Duyệt v (vd: in v ra màn hình)**
      - *Đánh dấu v đã duyệt*
      - Đưa v vào hàng đợi

- **Giải thuật (v2):**
  - Đưa 1 đỉnh **s** bất kỳ vào hàng đợi (vd: đỉnh 1)
  - Đánh dấu **s** đã được duyệt
  - **while** hàng đợi chưa rỗng **do**
    - Lấy đỉnh ở đầu hàng đợi ra ⇒ gọi là đỉnh u
    - **Duyệt u (vd: in u ra màn hình)**
    - **for** các đỉnh kề v của u **do**
      - **if** (v chưa được duyệt)
        - *Đánh dấu v đã duyệt*
        - Đưa v vào hàng đợi

*** Duyệt theo sâu (DFS)**

- Sử dụng ngăn xếp
- Các đỉnh trong ngăn xếp các đỉnh *sẽ được duyệt*. Một đỉnh có thể có mặt *nhiều lần* trong ngăn xếp.

- **Giải thuật (v1 – ngăn xếp):**
  - Đưa 1 đỉnh s bất kỳ vào ngăn xếp (vd: đỉnh 1)
  - **while** ngăn xếp chưa rỗng **do**
    - Lấy đỉnh ở đầu ngăn xếp ra ⇒ gọi là đỉnh u
    - **if** (u đã duyệt) **continue**; // bỏ qua
    - **Duyệt u (vd: in u ra màn hình)**
    - *Đánh dấu u đã duyệt*
    - **for** các đỉnh kề v của u **do**
      - Đưa v vào ngăn xếp

- **Giải thuật (v2 – đệ quy):**
  - Không sử dụng stack, chỉ sử dụng đệ quy

```
void dfs(u) {
      if (u đã duyệt) return;
      // Duyệt u: in u ra màn hình hoặc đánh số đỉnh u
      Đánh dấu u đã duyệt
      for (các đỉnh kề v của u) do
            dfs(v); // gọi đệ quy duyệt các đỉnh kề v của u
}
```

-> Để 2 phương pháp stack và đệ quy có thứ tự duyệt giống nhau:

- for các đỉnh kề v của u do
    - Đưa v vào ngăn xếp

- Ta phải xét qua các đỉnh kề v theo thứ tự **NGƯỢC LẠI** với thứ tự của vòng lặp trong giải thuật đệ quy

TÍNH LIÊN THÔNG TRONG ĐỒ THỊ

• **Đường đi (walk)**: đường đi chiều dài k đi từ đỉnh u = v_0 đến đỉnh v_k = v là danh sách các đỉnh và cung xen kẽ nhau:

- $v_0, e_1, v_1, e_2, v_2, e_3, \ldots, e_k, v_k$
- Cung e_i có đỉnh đầu là v_{i-1} và đỉnh cuối v_i

• **Đường đi đơn cung (trail)**: là đường đi các cung đều khác nhau.

• **Đường đi đơn đỉnh (path)**: là đường đi các đỉnh đều khác nhau.

• Đường đi đơn cung (trail) có đỉnh đầu và đỉnh cuối trùng nhau gọi là một **đường vòng (circuit)**

• Một đường vòng không có đỉnh nào lặp lại gọi là **chu trình (cycle)**.

• Chiều dài của walk, trail, path, circuit hay cycle là số cung của nó.

Liên thông trong đồ thị

Định nghĩa:

• Đồ thị vô hướng G được gọi là **liên thông** nếu và chỉ nếu với mọi cặp đỉnh u, v ∈ V luôn tồn tại đường đi (walk) từ u → v. Ngược lại, G được gọi là **không liên thông**.

• Đỉnh u được gọi là liên thông với đỉnh v ⇔ tồn tại đường đi từ u → v. Quan hệ liên thông trên G là tập các cặp có thứ tự (u, v) sao cho u liên thông với v.

• **Định lý:**

- Quan hệ liên thông là một quan hệ tương đương

• **Bổ đề:**

- Mỗi đường đi (walk) đi từ u đến v luôn chứa 1 một đường đi đơn đỉnh (path) đi từ u đến v.

Bộ phận liên thông

• Các bộ phận liên thông của đồ thị G là tập các đồ thị con liên thông lớn nhất của G (là các lớp tương đương của quan hệ liên thông).

• Đỉnh cô lập (có bậc bằng 0) cũng là một bộ phận liên thông chỉ gồm chính nó và được gọi là bộ phận liên thông tầm thường (trivial connected component).

Khoảng cách giữa 2 đỉnh

• **Định nghĩa:** Trên một đồ thị vô hướng, khoảng cách từ đỉnh u đến đỉnh v, ký hiệu d(u, v) được định nghĩa bằng:

- 0, nếu u = v
- Chiều dài đường đi ngắn nhất từ u đến v, nếu tồn tại đường đi từ u đến v.
- ∞, nếu không có đường đi từ u đến v.

• **Đường kính** của đồ thị vô hướng là khoảng cách lớn nhất giữa 2 đỉnh trên đồ thị.

Giải thuật kiểm tra tính liên thông

• Áp dụng giải thuật duyệt đồ thị để đánh số/đánh dấu (gán nhãn) các đỉnh

• Nếu sau khi duyệt tất cả các đỉnh đều có nhãn

=> liên thông, ngược lại không liên thông.

Liên thông (đồ thị có hướng)

• Cho đồ thị **có hướng** G = ⟨V, E⟩

• G được gọi là **liên thông yếu** ⇔ đồ thị vô hướng nền của nó liên thông (xem tính liên thông của đồ thị vô hướng)

• G được gọi là **liên thông mạnh** ⇔ giữa hai đỉnh x, y bất kỳ, luôn có đường đi từ x đến y.

• **Bộ phận liên thông mạnh**

• Đồ thị con liên thông mạnh: có đường đi giữa hai đỉnh bất kỳ.

• Đồ thị có hướng không liên thông mạnh bao gồm nhiều bộ phận liên thông mạnh

Giải thuật Tarjan – Tìm các bộ phận liên thông mạnh

*** Nhận xét:** Các đỉnh trong một chu trình liên thông với nhau.

Thuật toán Tarjan:

*** Các biến hỗ trợ:**

• **S**: stack lưu các đỉnh chưa tìm được BPLT mạnh

• **on_stack[v]**: kiểm tra v còn trên stack không

• **num[v]**: chỉ số của đỉnh v trong quá trình duyệt

• **min_num[v]**: chỉ số nhỏ nhất trong các chỉ số của các đỉnh trong stack S mà v đi đến nó được.

• **idx**: chỉ số dùng để gán cho num của các đỉnh (tăng dần)

*** Giải thuật:**

• Áp dụng **DFS** (đệ quy hoặc không đệ quy) để đánh số các đỉnh.

• Để tìm được chu trình, với mỗi đỉnh v, ngoài num[v], ta lưu thêm min_num[v] (là chỉ số nhỏ nhất trong các đỉnh có thể đi đến được từ v). Trong quá trình duyệt, min_num[v] sẽ được cập nhật.

• Khi duyệt xong 1 đỉnh, nếu num[v] = min_num[v] thì v là đỉnh bắt đầu (đỉnh gốc/đỉnh khớp) của bộ phận liên thông mạnh.

*** Lưu ý:**

• Thứ tự các đỉnh kề của 1 đỉnh được sắp xếp từ bé đến lớn.

• Khi duyệt xong 1 đỉnh y, quay về đỉnh cha x (đỉnh trước), cập nhật lại min_num[x] (so với **min_num[y]**)

• Khi xét 1 đỉnh kề y của x mà y đang có mặt trong stack ⇒ cập nhật lại min_num[x] (so với **num[y]**).

Cây duyệt theo chiều sâu

• Quá trình duyệt cây kết hợp với đánh số (num, min_num) tạo ra cây "duyệt đồ thị"

- Phần tiếp theo minh hoạ quá trình chạy giải thuật và cây "duyệt đồ thị tương ứng"
- Cung liên nét (cung thuận)
- Cung không liên nét: cung quay lui (minh hoạ việc cập nhật min_num)

DUYỆT ĐỒ THỊ & ỨNG DỤNG

Dựng cây duyệt đồ thị

- Trong quá trình duyệt đồ thị, có 2 bước chính
  - Duyệt đỉnh u
  - Xem xét duyệt các đỉnh kề v của nó
- Mối quan hệ "cha – con" giữa các đỉnh => cây duyệt đồ thị

Dựng cây duyệt đồ thị

- Tổ chức hàng đợi/ngăn xếp có khả năng lưu:
  - Đỉnh sắp sửa duyệt và đỉnh cha của nó (đỉnh đưa nó vào hàng đợi/ngăn xếp)
- Duyệt DFS đệ quy
  - Thêm một đối số p vào hàm DFS

int parent[MAX_N];

```
void DFS(int u, int p) {
    if (u đã duyệt)
        return;
    //Duyệt u, đánh dấu u đã duyệt
    //Cho p là cha của u
    parent[u] = p;

    for (các đỉnh kề v của u)
        DFS(v, u);
}
```

```
void DFS(int u, int p) {
    //Duyệt u, đánh dấu u đã duyệt
    //Cho p là cha của u
    parent[u] = p;

    for (các đỉnh kề v của u)
        if (v chưa duyệt)
            DFS(v, u);
}
```

Tìm đường đi trên đồ thị

- Tìm đường đi từ s đến t
  - Duyệt đồ thị từ đỉnh s và dựng cây trong quá trình duyệt
- Nếu trong quá trình duyệt, đỉnh t được duyệt => Có đường đi
  - Từ t lần ngược lại theo parent[t], parent[parent[t]] ... sẽ đến s.
- Nếu t không được duyệt => KHÔNG có đường đi từ s đến t.

Kiểm tra đồ thị chứa chu trình

- Thuật toán kiểm tra đồ thị chứa chu trình
  - Trạng thái của đỉnh:

- Chưa duyệt: chưa đụng tới nó (TRẮNG/WHITE)
- Đang duyệt: đã duyệt nó nhưng chưa duyệt hết kề của nó (XÁM/GRAY)
- Đã duyệt xong: duyệt nó và đã duyệt hết các kề của nó (ĐEN/BLACK)

- Thuật toán:
  - DFS + tô màu các đỉnh trong quá trình duyệt
  - Nếu khi xét 1 đỉnh kề v của u mà v có màu xám => tạo chu trình

• Kiểm tra đồ thị có HƯỚNG chứa chu trình

```
void DFS(u) {
    color[u] = GRAY;          //1. Đang duyệt u

    for (các đỉnh kề v của u) //2. Xét kề
        if (v có màu trắng)   //2a. v chưa duyệt
            DFS(v);
        else if (v có màu xám) //2b. v còn đang duyệt
            has_cycle = 1;     //tạo chu trình
        //2c. v đã duyệt xong, bỏ qua

    color[u] = BLACK;         //3. Duyệt xong u
}
```

• Kiểm tra đồ thị VÔ HƯỚNG chứa chu trình

```
void DFS(u, p) {
    color[u] = GRAY;                //1. Đang duyệt u

    for (các đỉnh kề v của u)       //2. Xét kề
        if (v == p) {               //2a. v là cha của p
            //bỏ qua
        } else if (v có màu trắng)  //2b. v chưa duyệt
            DFS(v, u);              //duyệt v với u là cha của v
        else if (v có màu xám)      //2c. v đang duyệt
            is_cycle = 1;           //tạo chu trình
        //2d. v đã duyệt xong (KHÔNG có)

    color[u] = BLACK;               //3. Duyệt xong u
}
```

Kiểm tra đồ thị phân đôi

• Đồ thị phân đôi (đồ thị hai phần, đồ thị hai bên, đồ thị hai phía, ...)
  • Đồ thị VÔ HƯỚNG
  • Có thể chia các đỉnh của đồ thị thành hai phần (không giao nhau) sao cho các đỉnh trong mỗi phần KHÔNG kề nhau

- Thuat toan:

• Có 3 trường hợp xảy ra khi xét v
  • v chưa có màu => tô màu ngược lại với màu của u
  • v có màu GIỐNG với u => KHÔNG TÔ ĐƯỢC
  • v có màu KHÁC với u => bỏ qua

• Màu "ngược lại"
  • Nếu đặt S = XANH + ĐỎ
  • Thì
    • Ngược của XANH = ĐỎ = S - XANH
    • Ngược của ĐỎ = XANH = S - ĐỎ

Code:

```
#define NO_COLOR -1
#define BLUE 1
#define RED 2

void colorize(u, c) {

   color[u] = c;                    //1. Đang duyệt u


   for (các đỉnh kề v của u)            //2. Xét kề

      if (v == NO_COLOR) {              //2a. v chưa có màu

         colorize(v, 3 - c);          //tô màu màu ngược với c

      } else if (color[v] == color[u])  //2b. màu v giống màu u

         conflict = 1;                //đụng độ, không tô được

      //2c. Màu khác nhau, bỏ qua

}
```

ĐỒ THỊ EULER & ĐỒ THỊ HAMILTON

Đồ thị Euler

- **Chu trình Euler** (Euler circuit) trong đồ thị G là một chu trình đơn cung chứa tất cả các cung của G
- **Đường đi Euler** (Euler path) trong đồ thị G là đường đi đơn cung chứa qua tất các đỉnh của G
- **Định lý Euler:**
  + Đa đồ thị (vô hướng) liên thông có ít nhất 2 đỉnh có **chu trình Euler** khi và chỉ khi tất cả các đỉnh của có bậc chẵn.
  + Đa đồ thị (vô hướng) liên thông có **đường đi Euler** (nhưng không có chu trình Euler) khi và chỉ khi nó 2 đỉnh bậc lẻ.

Thuật toán Fleury xây dựng chu trình Euler

- Chọn 1 đỉnh bất kỳ
- Chọn tiếp các cung liên tiếp cho đến khi tạo thành chu trình. Mỗi khi một cung được chọn, xóa nó khỏi đồ thị
- Các cung được chọn sao cho đỉnh đầu của cung mới là đỉnh cuối của cung cũ VÀ nó không phải là cầu (cầu: xóa bỏ cung này sẽ làm đồ thị mất liên thông) trừ phi không có lựa chọn khác.

Thuật toán đồ thị Euler

*** Cài đặt đệ quy:**

```
void findEulerPath (u) {
        for (các đỉnh kề v của u)
                xóa cung (u, v)
                findEulerPath (v);
        Thêm u vào path
}
```

*** Cài đặt dùng ngăn xếp:**

S: stack

```
void findEulerPath(u) {
        while (S != rỗng) {
            Lấy phần tử trên đỉnh S, gọi nó là u
            if (u không còn đỉnh kề)
                Thêm u vào path
            for (v = 1; v <= n; v++)
                if (v kề với u) {
                    xóa cung (u, v)
                    push v vào S
                    break; // Chỉ cần lấy 1 kề đầu tiên
```

```
            }
        }
    }
```

Ứng dụng

- Đi qua các con đường
- Giao hàng
- Chinese postman problem

Đồ thị Hamilton

Chu trình Hamilton (Hamilton circuit): Chu trình đơn đỉnh (simple circuit) đi qua các đỉnh của đồ thị, mỗi đỉnh đúng 1 lần (trừ đỉnh xuất phát)

Đường đi Hamilton (Hamilton path): Đường đi đơn đỉnh (simple path) đi qua các đỉnh của đồ thị G, mỗi đỉnh đúng 1 lần

Định lý Dirac:
Nếu G là đơn đồ thị có số đỉnh $n \geq 3$ và bậc của các đỉnh đều $\geq n/2$ thì G có chu trình Hamilton

Định lý Ore:
Nếu G là đơn đồ thị có số đỉnh $n \geq 3$ và $\deg(u) + \deg(v) \geq n$ với mọi cặp đỉnh không kề nhau thì G có chu trình Hamilton.

Ứng dụng

- Xếp tour du lịch
- Bài toán người giao hàng (Traveling Salesman Problem – TSP)
- Mã Gray

ĐƯỜNG ĐI NGẮN NHẤT

Biểu diễn đồ thị có trọng số

- Nếu có (u, v) thì **L[u, v] = trọng số của cung (u, v)**
- Nếu không có (u, v) thì **L[u, v] = NO_EDGE** (vd: -1)

Bài toán đường đi ngắn nhất

- Chiều dài/Chi phí (cost) của đường đi (path) là tổng các trọng số của các cung trên đường đi.

- Đường đi ngắn nhất (shortest path) là đường đi có chiều dài nhỏ nhất.

- Một đoạn đường đi (sub-path) của đường đi ngắn nhất cũng là đường đi ngắn nhất.

  - Tính chất cấu trúc con tối ưu (optimal substructure)
    => có thể áp dụng kỹ thuật quy hoạch động để thiết kế các giải thuật tìm đường đi ngắn nhất.

Thuật toán Moore-Dijkstra

- **Điều kiện áp dụng:**
  - Đồ thị có trọng số không âm
  - Có hướng hoặc vô hướng đều được

- **Ý tưởng chính:**
  - Khởi tạo đường đi trực tiếp.
  - Lần lượt cập nhật lại đường đi nếu **tìm được đường đi mới tốt hơn đường đi cũ.**

- Gọi đỉnh bắt đầu là s

- **Các biến hỗ trợ**
  - pi[u]: **chiều dài đường đi ngắn nhất** (tạm thời) từ s đến u.
  - p[u]: **đỉnh trước** đỉnh u trên đường đi ngắn nhất (tạm thời) từ s đến u.
  - mark[u]: đánh dấu đỉnh u (đã tìm được đường đi ngắn nhất đến u)

- Nếu mark[u] == 1 thì pi[u] chứa chiều dài đường đi ngắn nhất từ s đến u và p[u] là đỉnh trước của đỉnh.

- **Khởi tạo:**
  - pi[u] = oo ∀ u
  - p[u] = -1 ∀ u
  - mark[u] = 0 ∀ u
  - pi[s] = 0; đường đi ngắn nhất từ s -> s bằng 0.

- Lặp n-1 lần
  - Chọn đỉnh **chưa đánh dấu** và có **pi[u] nhỏ nhất**
  - Đánh dấu u

• Xét các đỉnh kề v (chưa đánh dấu) của u để cập nhật đường đi nếu đường đi qua u rồi đến v tốt hơn đường đi cũ.

Thuật toán Bellman-Ford

• Điều kiện áp dụng:

- Đồ thị có trọng số âm hoặc không âm
- Có hướng hoặc vô hướng đều được
- Không áp dụng nếu có chu trình âm đi tới được từ đỉnh bắt đầu

• Ý tưởng chính:

- Khởi tạo đường đi ban đầu từ đỉnh bắt đầu đến các đỉnh khác.
- Lần lượt xét tất cả các cạnh để cập nhật lại đường đi nếu tìm được đường đi mới tốt hơn đường đi cũ.
- Sau n-1 lần cập nhật, ta thu được đường đi ngắn nhất nếu đồ thị không có chu trình âm.

• Gọi đỉnh bắt đầu là s

• Các biến hỗ trợ

- pi[u]: **chiều dài đường đi ngắn nhất** (tạm thời) từ s đến u.
- p[u]: **đỉnh trước** đỉnh u trên đường đi ngắn nhất (tạm thời) từ s đến u.

• Nếu sau n-1 lần lặp mà không còn cập nhật được nữa thì pi[u] là chiều dài đường đi ngắn nhất từ s đến u và p[u] là đỉnh trước của u trên đường đi đó.

- **Khởi tạo:**
  - pi[u] = oo ∀ u
  - p[u] = -1 ∀ u
  - pi[s] = 0; đường đi ngắn nhất từ s -> s bằng 0.
- Lặp n-1 lần
  - Xét tất cả các cạnh (u, v) của đồ thị
  - Nếu pi[u] + w(u, v) < pi[v] thì cập nhật:
    - pi[v] = pi[u] + w(u, v)
    - p[v] = u
- **Kiểm tra chu trình âm:**
  - Xét lại tất cả các cạnh (u, v)
  - Nếu còn tồn tại cạnh sao cho pi[u] + w(u, v) < pi[v]
    - Kết luận đồ thị có chu trình âm
    - Không tồn tại đường đi ngắn nhất

Thuật toán Floyd–Warshall

- **Điều kiện áp dụng:**
  - Đồ thị có trọng số âm hoặc không âm
  - Có hướng hoặc vô hướng đều được
  - Dùng để tìm đường đi ngắn nhất giữa mọi cặp đỉnh
  - Không áp dụng nếu đồ thị có chu trình âm

- **Ý tưởng chính:**
  - Khởi tạo khoảng cách trực tiếp giữa mọi cặp đỉnh.
  - Lần lượt chọn từng đỉnh k làm đỉnh trung gian.
  - Cập nhật lại đường đi từ i đến j nếu đi qua k tốt hơn đường đi cũ.

- **Gọi:**
  - d[i][j]: **chiều dài đường đi ngắn nhất** (tạm thời) từ i đến j
  - p[i][j]: **đỉnh trung gian** hoặc **đỉnh trước** dùng để truy vết đường đi từ i đến j
- Nếu sau khi xét hết các đỉnh trung gian, d[i][j] không còn thay đổi thì d[i][j] là chiều dài đường đi ngắn nhất từ i đến j.

- **Khởi tạo:**
  - d[i][j] = w(i, j) nếu có cạnh từ i đến j
  - d[i][j] = oo nếu không có cạnh từ i đến j
  - d[i][i] = 0 ∀ i
  - p[i][j] = -1 nếu chưa có đường đi trực tiếp
- Lặp với k từ 1 đến n
  - Lặp với i từ 1 đến n
    - Lặp với j từ 1 đến n
      - Nếu d[i][k] + d[k][j] < d[i][j] thì cập nhật:
        - d[i][j] = d[i][k] + d[k][j]
        - p[i][j] = k

- **Kiểm tra chu trình âm:**
  - Nếu tồn tại đỉnh i sao cho d[i][i] < 0
    - Kết luận đồ thị có chu trình âm

THỨ TỰ TOPO & ỨNG DỤNG

Thứ tự Topo

- Thứ tự topo (Topological order) của các đỉnh trên đồ thị có hướng G là một cách xếp thứ tự các đỉnh của G sao cho với mọi cung (u, v), đỉnh gốc u phải đứng trước đỉnh ngọn v.

- Sắp xếp topo (topological ordering/sort): sắp xếp các đỉnh theo thứ tự topo

- Đồ thị có hướng không chu trình ((Directed Acyclic Graph - DAG):

+ Có hướng
+ Không chứa chu trình
+ Tồn tại ít nhất 1 thứ tự topo

- Thuật toán sắp xếp Topo:

*** BFS:**

- **Khởi tạo**

  Tính bậc vào của các đỉnh, d[u]
  Q chứa các đỉnh có bậc vào = 0
  L = rỗng

- **Lặp**

```
while (Q khác rỗng)
    Lấy phần tử đầu hàng đợi Q, gọi nó là u
    Thêm u vào L
    for các đỉnh kề v của u
        Giảm bậc vào của đỉnh v
        if bậc vào của v = 0
            Thêm v vào Q
```

*** DFS (đệ quy):**

```
void topo sort(Graph *pG, int u, List *pL){
    mark[u] = 1;
    for(int v = 1; v <= pG->n; v++)
        if(pG->A[u][v] && !mark[v]){
            topo_sort(pG, v, pL);
            d[v]--;
        }
    push_back(pL, u);
}
```

Xếp hạng các đỉnh trong đồ thị

- Gốc/nguồn (root/source) của đồ thị

  + Đỉnh không có cung đi vào (bậc vào = 0)
  + DAG có ít nhất 1 gốc

- Hạng (rank) của 1 đỉnh:

  + u là gốc, rank[u] = 0
  + ngược lại, rank[u] = số cung trên *đường đi dài nhất* từ 1 gốc đến u.

Thuật toán xếp hạng đồ thị (dựa trên BFS):

• Ý tưởng:

- Xếp hạng cho (các) gốc (hạng 0)
- Loại bỏ gốc ra khỏi đồ thị => xuất hiện gốc mới
- Xếp hạng cho gốc mới (hạng 1)
- Loại bỏ gốc ra khỏi đồ thị => xuất hiện gốc mới
- ...

• Nhận xét:

- Xoá đỉnh u: giảm bậc vào của các đỉnh kề của u
- Đỉnh kề nào có bậc vào giảm về 0 => gốc mới

• Cài đặt

- Các biến hỗ trợ
  - d[u]: bậc vào của đỉnh u
  - S[k]: danh sách các gốc ở bước k
  - rank[u]: hạng của đỉnh u
  - k: bước lặp

Khởi tạo

```
Tính bậc vào của các đỉnh: d[u]
Đưa các gốc vào S[0]
k = 0

while (S[k] != rỗng)
  //1. Làm rỗng danh sách S[k+1]
  //2. Xếp hạng và giảm bậc của đỉnh kề
  for (các đỉnh u trong danh sách S[k])
    rank[u] = k /* Xếp hạng cho u*/
    for (các đỉnh kề v của u)
      d[v]-- /*Xóa u (giảm bậc vào của các đỉnh kề của u*/
      if (d[v] == 0)
        đưa v vào S[k+1]
  k++
```

Quản lý dự án

- **Mô hình hoá về đồ thị**
  - Công việc (CV) => Đỉnh
  - Sự phụ thuộc giữa 2 CV => Cung
  - d[u]: thời gian hoàn thành CV u => trọng số của đỉnh u

  - Thêm 2 đỉnh: α, β (tương ứng với 2 CV giả)
  - Thêm cung nối α với các đỉnh có bậc vào bằng 0
    - d[α] = 0
  - Thêm cung nối các đỉnh có bậc ra bằng 0 với β

- Người quản lý dự án (project manager)
  - Tổ chức thực hiện công việc như thế nào để
    - Thoả mãn sự phụ thuộc của các công việc
    - Thời gian hoàn thành dự án ngắn/ít nhất

- **Thời điểm sớm nhất bắt đầu các công việc:** t[u]
  - t[β]: thời điểm sớm nhất hoàn thành dự án

- Tính t[u]
  - t[α] = 0
  - Xét các đỉnh u theo thứ tự hạng **tăng dần**
    - **t[u] = max {t[x] + d[x]}** với x là đỉnh tương ứng với công việc trước của công việc u.

- **Thời điểm trễ nhất bắt đầu các công việc mà không ảnh hưởng đến tiến độ dự án:** T[u]

- Tính T[u]
  - T[β] = t[β]
  - Xét các đỉnh u theo thứ tự hạng **giảm dần**
    - **T[u] = min {T[v]} - d[u]** với v là đỉnh kề của u.

- **Công việc then chốt**
  - t[u] = T[u]
  - Đường nối các công việc then chốt: đường then chốt (critical path)

CÂY & CÂY KHUNG TỐI THIỂU

Cây (tree)

*** Định nghĩa:**

- **Cây (tree):** đồ thị vô hướng liên thông và không có chu trình.

- **Rừng (forest):** đồ thị vô hướng không có chu trình.

- **Đồ thị liên thông tối thiểu (minimally connected):**

    + Đồ thị liên thông và nếu xoá một cung bất kỳ thì nó không còn liên thông nữa.

    + Đồ thị liên thông tối thiểu không chứa chu trình

*** Định lý:**

1/ Đồ thị G là cây ⇔giữa mỗi cặp đỉnh của G có đúng 1 đường đi (path).

2/ Cây có n đỉnh sẽ có n – 1 cung.

3/ Đồ thị liên thông n đỉnh, n – 1 cung là 1 cây

4/ Đồ thị G là cây ⇔G liên thông tối thiểu

- <= : G liên thông tối tiểu => G không chứa chu trình => G là cây

- => : G là cây => G không chứa chu trình => xóa 1 cung bất kỳ sẽ làm G không còn liên thông nwuxa => G liên thông tối tiểu.

5/ Đồ thị G có n đỉnh, n-1 cung, không chứa chu trình thì liên thông.

6/ Cây có ít nhất 2 đỉnh thì sẽ có ít nhất 2 đỉnh treo (đỉnh bậc 1)

7/ (rừng): 1 rừng n đỉnh, k cây sẽ có n – k cung.

Note: Nếu một cây thêm 1 cung bất kỳ -> tạo chu trình; bớt 1 cung -> không liên thông

Cây khung, cây khung nhỏ nhất:

*** Cây khung:**

- Cây khung (spanning tree): cây bao trùm/cây phủ

    + G =<V, E>là đồ thị vô hướng liên thông

    + Đồ thị con T = <V, E'> là cây khung của G:

        . T là cây

        . Chứa tất cả các đỉnh của G, Có |V| - 1 cung

- Đồ thị (liên thông) có thể có nhiều cây khung.

*** Cây khung nhỏ nhất:**

- Minimum spanning tree (MST)

    + Cây khung tối thiểu/cây khung nhỏ nhất

    + Cây khung có trọng lượng nhỏ nhất

…

- G = <V, E> là đồ thị vô hướng liên thông. Cung (u, v) có trọng số w(u, v)

- Cây T là cây khung tối tiểu của G nếu:

+ T là cây khung của G

+ T có tổng trọng số nhỏ nhất trong tất cả các cây khung của G

Thuật toán Krusal

1. Sắp xếp các cung của G theo thứ tự trọng số tăng dần

2. Khởi tạo cây T rỗng (không chứa cung nào cả)

- for (các đỉnh u của G)
  - parent[u] = u; //Mỗi đỉnh là 1 BPLT

3. Lần lượt xét từng cung của G (theo thứ tự đã sắp xếp)

  - r_u = find_root(u); //r_u là gốc của u
  - r_v = find_root(v); //r_v là gốc của v
  - if (r_u != r_v) //2 BPLT khác nhau => không tạo CT
    - Thêm cung (u, v) vào T
    - parent[r_v] = r_u; //cho gốc của v làm con của gốc của u

(+ Nếu thêm cung e = (u, v) vào T mà không tạo thành chu trình thì thêm e vào T

+ Ngược lại, bỏ qua cung e)

- Điều kiện dừng:

T có n – 1 cung hoặc tất cả các cung của G đều đã được xét

* Ứng dụng: Bài toán xây dựng, thiết kế mạng…

Thuật toán Prim

Xây dựng cây khung:

- Khởi tạo cây T rỗng

- Chọn 1 đỉnh u bất kỳ làm đỉnh bắt đầu, đánh dấu nó đã xét (mark[u] = 1), các đỉnh khác đều chưa xét.

- Lặp n - 1 lần

+ Tìm đỉnh v chưa xét, gần với 1 trong các đỉnh u đã xét nhất

+ Thêm cung (u, v) vào T

+ Đánh dấu v đã xét

Biểu diễn đồ thị bằng ma trận trọng số

```
typedef struct {
  int n, m;
```

```
  int L[100][100];
} Graph;
```

Các biến hỗ trợ

- **mark[u]**: đỉnh u đã xét (1) hay chưa (0)
- **pi[u]**: khoảng cách ngắn nhất từ u đến 1 trong các đỉnh đã xét
- **p[u]**: đỉnh đã xét gần với u nhất

Khởi tạo

- Với mọi u, mark[u] = 0; π[u] = oo
- mark[1] = 1
- for (các đỉnh kề v của 1)
    - π[v] = L[1][v]; //khoảng cách từ v đến 1 = TS cung (1, v)
    - p[v] = 1; //1 là đỉnh đã xét gần với v nhất
- Lặp n – 1 lần
    - Chọn đỉnh v chưa xét, có **π[v] nhỏ nhất**
    - mark[v] = 1; //Đánh dấu v đã xét
    - Thêm cung (p[v], v) vào T
    - for (các đỉnh kề x chưa xét của v)
        - Cập nhật lại π[x] và p[x] (nếu L[v][x] < π[x])

CÂY CÓ HƯỚNG & CÂY KHUNG TỐI THIỂU

Cây có hướng

*** Định nghĩa:** Đồ thị có hướng G = <V, E> là một cây có hướng, gốc r khi và chỉ khi:

+ G không có chu trình vô hướng

+ Luôn có đường đi từ r đến các đỉnh khác

*** Tính chất:** G là cây có hướng gốc r

1. Tồn tại đỉnh r được nối với mỗi một đỉnh khác bằng một đường đi duy nhất xuất phát từ r.
2. Gần liên thông mạnh và cực tiểu đối với tính chất này.
3. Liên thông và tồn tại một đỉnh r có bậc trong bằng không và bậc trong của những đỉnh khác r là bằng 1.
4. Không có chu trình và tồn tại một đỉnh r có bậc trong bằng không và bậc trong của những đỉnh khác r là bằng 1.
5. Gần liên thông mạnh và không có chu trình.
6. Gần liên thông mạnh và có n-1 cung.

Cây khung có hướng

*** Cây khung có hướng:**

Cây khung của đồ thị G:

+ Cây có hướng.

+ Gồm tất cả các đỉnh của đồ thị G.

*** Cây khung có hướng nhỏ nhất:**

Là cây khung có tổng số trọng số các cung nhỏ nhất.

Thuật toán Chu-liu/Edmonds

- Pha co:

+ Gọi đồ thị gốc là G_0, t = 0

+ Lặp

- **Xây dựng đồ thị xấp xỉ H_t từ G_t**
  Trừ gốc ra, với mỗi đỉnh còn lại giữ lại 1 cung đi đến nó có trọng số nhỏ nhất (bỏ các cung khác đi)
- Nếu H_t không chứa chu trình => **thoát vòng lặp chuyển sang pha giãn**
- Ngược lại co G_t thành G_{t+1}
- t = t + 1

- Pha giãn:

Giãn cây khung T_{t+1} thành cây khung T_t của đồ thị G_t

+ Mở đỉnh (được gom lại trong pha co) => chu trình

+ Điều chỉnh trọng số của cung đi đến chu trình

+ Xoá bỏ 1 cung trong chu trình

Giãn H_{t+1} thành cây khung của đồ thị G_t

Ứng dụng:

Xây dựng hệ thống dẫn nước.

LUỒNG CỰC ĐẠI TRONG MẠNG

Mạng (các luồng) & luồng trong mạng

- Network (of flows)/Flow network

- Mạng là 1 bộ 5: N = <V, E, s, t, c> trong đó

    + <V, E> là đồ thị có hướng biểu diễn cho mạng

        + V: đỉnh/nút

        + E: cung

    + s: đỉnh phát (source)

    + t: đỉnh thu (sink)

    + c: (u,v) -> c(u,v) hàm mô tả khả năng thông qua của 1 cung

        c(u,v): luồng lớn nhất có thể đi qua cung (u, v)

Luồng (trong mạng)

Luồng

- (Network) Flow: thứ lưu thông trên mạng

    - Một luồng đi từ đỉnh phát (s) đến đỉnh thu (t) là 1 hàm f: (u, v) -> f(u,v)

    - thoả mãn các điều kiện:

        + Với mỗi cung: 0 <= f(u,v) <= c(u,v)

        + Với mỗi đỉnh khác s và t: tổng luồng vào = tổng luồng ra

        + Tổng luồng ra khỏi s = tổng luồng vào t

    - **Giá trị luồng |f| = tổng luồng ra khỏi s**

- Mẹo: luồng s-t là cách gán các **giá trị luồng cho từng cung** của mạng sao cho **thoả mãn điều kiện luồng**.

Luồng cực đại

• Cho mạng N = <V, E, s, t, c>, tìm luồng f (từ s đến t) có **giá trị luồng |f| lớn nhất**

Lát cắt

- Một lát cắt cát tách s và t (s-t cut)

    -> Là một cách chia (phân hoạch) tập đỉnh V thành hai phần rời nhau (S, T) sao cho s nằm trong S và t nằm trong T

- Các cung của một lát cắt (tách s và t): Tập các cung xuất phát từ S đi đến T

$$\{(u,v) \mid u \in S \text{ và } v \in T\}$$

- Khả năng thông qua của 1 lát cắt (tách s và t)

$$c(S,T) = \sum_{\{u \in S, v \in T\}} c(u,v)$$

-> Tổng khả năng thông qua của các cung của lát cắt

Các bổ đề về luồng và lát cắt

Bổ đề 1:

- Nếu gọi:

+ f là luồng đi qua mạng N

+ (S, T) là 1 lát cắt tách s và t

- thì

+ Luồng đi qua lát cắt = luồng ra khỏi s

= luồng đi vào t

$$\sum_{\textbf{e out of S}} f(e) - \sum_{\textbf{e in of S}} f(e) = \sum_{\textbf{e out of s}} f(e) = |f|$$

Bổ đề 2:

- Nếu gọi:

+ f là luồng đi qua mạng N

+ (S, T) là 1 lát cắt tách s và t

- thì

+ Giá trị luồng không vượt quá khả năng thông qua của lát cắt

$$|f| \leq c(S,T)$$

Hệ quả:

- Gọi:

+ f là luồng đi qua mạng N

+ (S, T) là 1 lát cắt tách s và t

- Nếu |f| = c(S, T) thì f là luồng lớn nhất (luồng cực đại) và (S, T) là lát cắt hẹp nhất.

Định lý luồng cực đại lát cắt hẹp nhất (Ford-Fulkerson, 1956)

-> Giá trị luồng cực đại = khả năng thông qua của lát cắt hẹp nhất.

Thuật toán//PP Ford-Fulkerson

• Ý tưởng: **Tìm lát cắt hẹp nhất** bằng cách **tăng luồng** đi qua mạng.

• Khởi 1 luồng hợp lệ (thường là luồng có giá trị 0)

• Tăng luồng đi qua mạng cho đến khi bị ngẽn => lát cắt hẹp nhất => giá trị luồng cực đại.

• **Tìm đường tăng luồng**:

- Tìm đường đi từ s đến t sao cho có thể tăng thêm luồng trên đường đi đó.
    - **QS1: Tìm đường đi từ s đến t sao cho c(e) > f(e) với tất cả các cung trên đường đi**

• Xây dựng đồ thị tăng luồng/thặng dư (residual graph)

+ Đường tăng luồng = đường đi từ s đến t trên đồ thị tăng luồng

+ Luồng cực đại ⇔ không còn đường tăng luồng

• **Đánh dấu** các đỉnh để tìm đường tăng luồng trên đồ thị tăng luồng

- Không cần xây dựng tường minh đồ thị tăng luồng
- Sử dụng **duyệt theo chiều sâu** để tìm đường đi từ s đến t.

• Mỗi đỉnh u được đánh dấu/gán nhãn với các thông tin sau:

- **d[u]**: hướng của cung (+: cung thuận, -: cung nghịch "undo" luồng)
- **p[u]**: đỉnh trước đỉnh u
- **o[u]**: lượng tăng luồng lớn nhất có thể tăng

• Khởi tạo f(u, v) = 0 với mọi cung (u, v)

• **while** (1) {
- **Tìm đường tăng luồng**
- Nếu không tìm thấy => thoát vòng lặp
- **Tăng luồng theo đường tăng luồng**

}

• Lát cắt hẹp nhất = (S, T)
- S: các đỉnh đã có nhãn
- T là các đỉnh chưa có nhãn

• Tìm đường tăng luồng từ s đến t

- Đánh dấu s: (+, s, oo)
- Đánh dấu các đỉnh kề của s
- Đánh dấu các đỉnh kề của đỉnh kề của s
- ...
- Cho đến khi t được đánh dấu => tìm được đường đi từ s đến t.

• Có thể cài đặt bằng đệ quy hoặc vòng lặp sử dụng stack

• Tìm đường tăng luồng từ s đến t dùng Stack

1. Đánh dấu s: (+, s, ∞)
2. Push s vào Stack
3. while (stack không rỗng) {

a. u = top(Stack); pop(Stack);

b. for (các đỉnh kề v của u và có **f(u, v) < c(u, v)**)
   Đánh dấu v: (+, u, min{**σ[u]**, c(u, v) − f(u, v)})
   Push v vào Stack

c. for (các đỉnh x có cung đi đến u có **f(x, u) > 0**)
   Đánh dấu x: (-, u, min{**σ[u]**, f(x, u)})
   push x vào Stack

d. Nếu t được đánh dấu => thoát vòng lặp while

}

• **Tăng luồng theo đường tăng luồng**

```
x = t;
sigma = σ[t]
while (x != s) {
   if (d[x] == '+') //Cung thuận
      f(p[x], x) += sigma; //TĂNG LUỒNG
   else            //Cung nghịch
      f(x, p[x]) -= sigma; //GIẢM LUỒNG
   x = p[x];
}
```

• Tất cả các cung trên đường tăng luồng sẽ được tăng/giảm 1 lượng giống nhau = **σ[t]**

• Giải thuật Ford-Fulkerson thường được gọi là **phương pháp** (method) Ford-Fulkerson thay vì **giải thuật** (algorithm)
  • Không mô tả rõ ràng cách tìm đường tăng luồng trên đồ thị tăng luồng

• Biến thể:
  • Giải thuật **Edmonds-Karp**
    • Tìm kiếm theo chiều rộng, sử dụng Queue thay vì Stack
  • Tìm đường tăng luồng có lượng tăng luồng nhiều nhất (cải biên Dijkstra)